Sesión ①

<%! int a = 0; %> Campo de instancia del servlet compartido. Con tres usuarios: a = 1,2,3
<% int b = 0; %> variable local de _jspService() (por petición) con 3 usuarios: b = 1,1,1

Porque el contenedor ~~crea~~ creo una sola instancia del servlet y la reutilizo para todos las peticiones, y se creo una servlet por eso a persiste entre usuarios y b reinicio en cada petición.

Sesión ②

Quedan 2 filas ambos con correo = null

Insert into usuarios (correo) values (null),(null);
→ Resultado: id = 1 correo = null, id = 2 correo = null

Explicación:

- Correo varchar (100) UNIQUE – la restricción UNIQUE no considera los null como los duplicados en sql, null es igual a otro null).
- Por eso los dos insert con correo=null no violan la restricción.
- Id INT AUTO_INCREMENT PRYMARY KEY → se genero solos 1 y 2.

Sesión ③

① • Consulta falla con un error en MySQL

Error 1054 (42S22): unknown column 'doble' in 'where clause'
Porque el alias doble se define en el select, pero el where se evalua antes que la lista de columnas ~~[illegible]~~ del select.
Por eso doble todavia no existe cuando se evalua el where MySQL no le reconoce.

Sesión (4)

Muestra 1000
Por que carlos tiene 2 canjes → el join genera dos filas de carlos, cada uno con puntos 500, y el SUM (e.puntos) suma 500+500 = 100 y el GROUP BY c.nombre agrupa pero no elimina la duplicación del join

Sesion 8 (5)

El usuario ve el texto literal <h1>Ranking</h1>; el HTML no se interpreta, se muestra como texto plano en la pagina por que Response.setContentType("Text/plain; charset=utf-8" le dice al navegador que es texto plano, no HTML".

- El navegador, al recibir text/plain, no renderiza las etiquetas muestra <h1>Ranking</h1> tal cual con los signos < y > visibles, no aparece un titulo grande "Ranking": aparece todo la cadena

sesion 8 (6)

Devuelve "/5"

- por que el mapeo es @webServlet("/juegos/*") -> y el * captura todo despues de /juego
- la url es el /juego /5.
- request.getPathInfo devuelve la parte de la ruta que quedo despues del patron del servlet incluyendo la barra inicial

Sesio 7 (7)

Lanza una ecepsión: SQLException: column 'nombre' not found.
Porque la consulta solo devuelve una sola columna, y ~~su~~ nombre ya no es nombre: El AS estudiante la renombra
- en el resultset, la columna se llama estudiante no nombre.
- rs.getString("nombre" busca una colum llamada nombre no existe error.

sesion 7 (8)

Luis sigue con 200 puntos en la base de datos (el update no se guardo).
Por que c.setAutoCommit(false) -> desactiva el guardado automatico desde ese momento, los cambios quedan en una transacción abierta no confirmada
- El update se ejecuto pero solo modifico la transacción pendiente, no la base de datos definitivo
- Al salir del Try-with-resources, connection.close() se llama automaticamente
- Como nunca se llamo a commit(), el driver hace roll back implicito al cerrar la conexión
- Resultado update se descarto. Luis quedo con 200 puntos.

Sesión 6 (9)

Se borra con solo abrir un enlace. Por que doGet deriva a doPost y un GET debería solo leer; el usuario para borrar, cualquier clic a recurso ejecuta el DELETE.

Sesión 6 (10)

Devuelve carlos mendoza. Porque: _a% pide un caracter cualquiera + 'a' + resto -> "carlos" cumple; "Ana" y "luis" no.

Sesión 7 (11)

Devuelve n=1. Porque: el WHERE encuentra la fila (existe el id 1) y connectorJ cuenta filas encontradas, no si el valor cambió.

Sesión 6 (12)

Ana no aparece. Porque: el LEFT JOIN la conservaría con null, pero el WHERE h.id_recompensa = 1 elimina esas filas null, así que se comporta como INNER JOIN.

Sesion 8 (13)

Devuelve null. Porque: el input tiene id pero no name; el navegador solo envía campos con name.

Sesion S7 (14)

Funciona sin error, Porque: CachedRowSet copia las filas a memoria con populate(); al cerrar la conexión los datos ya están guardados en el objeto

Sesion 8 (15)

Línea 1: null. Línea 2: vacío. Porque: <%= %> imprime el valor tal cual (null como texto), mientras que el ${param.nombre} convierte null en cadena vacía.

Sesion 8 (16)

Muestra Assassin, Porque: el apostrofo de Assassin's cierra el value= antes de tiempo y el resto del texto queda fuera del atributo.

Sesión 8 (17)

Sí se ejecuto el borrado. Por que: sendRedirect solo añade la cabecera de redirección; no detiene el metodo, así que la linea siguiente (dao.eliminar) corre igual. Falta return;.

sesión 7 (18)

Carlos quedo con 350 puntos y sin canje registrado. por que, con auto commit cada sentencia se confirma solo; el UPDATE se guardo, el inserte se guardo el INSERT fallo por fk y no revierte el UPDATE. falta una transacción.

Sesión 6 (19)

Fallo el create table con error.
Por que el ON DELETE SET NULL exige que la columna id_cat acepte null, pero esta declarado NOT NULL. MySQL rechazo la definición por incompatibilidad. (si fuera ON DELETE cascade, si funcionaria.)

sesión 7 (20)

Fallo con SQLException.
Por qué: Nombre es NOT NULL, y setString (1, null) intento insertar null en una columna obligatoria. la BD rechazo el insert.
Ademas, setString (1, null) en JDBC envio NULL SQL, no la cadena "null".

sesion 6 (21)

Devuel 3, 2. Porque: count(*) cuenta todas las filas (3). Count(puntos) cuenta solo valores no null (carlos 500 y luis 200; B. no es null) -2

sesion 7 (22)

Fallo en SQLException.
Por que: El PreparedStatement tiene dos ? (nombre y puntos) pero solo se asigno el primero. El segundo quedo sin valor JDBC lanzo error al ejecutar (executeUpdate).

Sesion 7

(23)

se insertan 3 filas (o falla, segun como se corrija) porque: el bucle agrega 3 bloques (x1, x2, x3), pero ps. setString (1,T) esta mal: pasa el array entero en lugar de n. Eso lanza error de tipo antes de llegar a executeBatch(). si se corrige a ps.setString (1, n), executeBatch() inserta 3 filas y r devuelve [1,1,1].

Sesion 7 (24)

rs.getString ("puntos") lanza SQLException (o devuelve error). Por que: se llama rs.getString (..) antes de rs.next(). El cursor del ResultSet esta antes de la primera fila; hay que llamar rs.next() primero. El orden correcto es: rs.next() -> rs.getString ("punto").

sesion 6 (25)

Devuelve solo Ana Torre (1 fila): Por que: puntos <> 500 es verdadero para Ana (Null<>500 es Null / desconocido... ojo, ver abajo).
MATIZ: en SQL Null <> 500 no es verdadero (es Null / desconocido), asi que Ana no deberia aparecer. La respuesta correcta es: devuelve 0 filas. Carlos (500) no cumple <> 500; Ana (Null) da desconocido y se descarta; Luis (200) si cumple -> solo Luis Rojas

Sesion 7 (26)

El login se salta: or = true.
Por que la concatenacion produce
select * from estudiante where nombre = '' or '1'='1'
'1'='1' siempre es verdadero -> devuelve todas las filas - rs.next()
da true -> login concedido sin credenciales validas. Es inyeccion SQL se evita con preparedStatement y ?

sesion 8 (27)

Llega con codigo HTTP 200 k
por que el servlet no llama a response.setStatus ni sendError solo escribe HTML en la respuesta. Aunque el mensaje diga "error", el codigo HTTP sigue siendo 200 (exito) por que no se cambio explicitamente. Para un error real habria que usar response.setStatus (404) o sendError (404, "...")

Sesión 6 (28)

Pepe → 0; Lolo → Null.
Por qué: Pepe omite puntos, y la columna tiene DEFAULT 0 se guardó 0. Lolo envió NULL explícito y como la columna admite null, se guardó NULL (el default solo aplica cuando se omite la columna, no cuando se envía Null.

sesión 6 (29)

Tomcat responde con HTTP 500 (internal server error).
Por qué: request.getParameter("d") devuelve "" (cadena vacía), Integer.parseInt("") lanza NumberFormatException, que no se captura → el servlet falla - 500. Si el parámetro no viniera, devolvería null y parseInt(null) también lanzaría error.

sesión 7 (30)

MySQL rechazó la conexión: la contraseña es incorrecta (o el usuario root no tiene esa password).
Por qué: access denied ... (using password: yes) significa que sí envió una contraseña pero no coincide con la del usuario root en localhost. El usuario existe, pero las credenciales son inválidas. (Si no se hubiera enviado las ~~credenciales~~ contraseñas, diría using password: NO

sesión 6 (31)

Falla con error de integridad referencial (FK): por que truncate table estudiante, table estudiantes no se puede ejecutar otra tabla (historial - canjes) tiene una foreign key que la referencia. MySQL lo bloquea. Habría que borrar primero los canjes (o usar Delete

Sesión 7 (32)

No se ordena por puntos; el orden es indefinido (normalmente por orden de inserción). Porque ORDER BY ? no sustituye un nombre de columna: el ? se trata como valor literal (la cadena "puntos"), no como identificador: la consulta queda como ORDER BY 'puntos' DESC, que ordena por una constante → sin efecto real. Para ordenar dinámicamente hay que concatenar el nombre columna válido (whitelist), no usar ?.

Sesión 6 ㉝

Devuelve tres juegos (Among Us, Minecraft, Gran Turismo 7). FIFA 26 no entra: Por que BETWEEN 15.5 AND 189 es inclusivo: incluye 15.50, 89,90 y 189,00. FIFA 26 vale 149.90 fuera del rango

Sesión 6 ㉞

Se insertó correcto 1 fila.

Porque los ids 1 existen en estudiantes y recompensas, así que las FK se cumplen. id_canje es AUTO_INCREMENT y fecha_canje toma CURRENT_TIMESTAMP por defecto. El INSERT es válido.

Sesión 7 ㉟

Imprime 0.3000000004 (no 0.3)
Por que: double no representa exactamente decimales; 0.10 + 0.20 acumula error de punto flotante. Para dinero hay que usar BigDecimal (rs.getBigDecimal("precio")).

Sesión 6 ㊱

La segunda línea lanza IllegalStateException.
Por que, un servlet solo se puede usar en getWriter() o getOutputStream(), no ambos. una vez que llamaste a getWriter(), getOutputStream() lanza excepción (getWriter() has already been called). Hay que elegir uno solo.

Sesión 7 ㊲

No, ya no es obligatorio. Por que ~~después~~ desde JDBC 4.0 (Java 6), el driver se carga automáticamente por el mecanismo SPI (META-INF/service (java.sql.Driver) si el .Jar está en el classpath. class.forName (...) es innecesario (aunque inofensivo). Solo era obligatorio en Java 5 y anteriores

Sesión 7 38

Lanza SQL exception (o getString devuelve error) porque no se llamó a rs.next antes. Porque String n = rs.getString(1) se ejecuta antes de rs.next(). El cursor está antes de la primera fila - exception. El orden correcto es if (rs.next()) { String n = rs.getString(1); }.
(además, WHERE 1=0 no devuelve filas así que rs.next() sería false.)

Sesión 7 ㊴

Luis quedó con puntos = null (o 0, según el DEFAULT).
Porque el preparedStatement es el mismo y conserva el valor anterior del segundo executeUpdate(). En JDBC, los parámetros persisten entre ejecuciones del mismo preparedStatement, así que el segundo ? sigue valiendo 10 → Luis quedaría ~~con~~ con 10

Sesión 8 ㊵

visitas++ se ejecuta sin control de concurrencia → conteo incorrecto (pierde incrementos). Porque: El contenedor crea una instancia del servlet y la comparte entre hilos, si dos peticiones hacen visitas++ a la vez ambos leen el mismo valor antes de escribir (race condition) y uno sobrescribe a lo otro. Con 2 usuarios simultáneos puedes terminar con 1 en vez de 2. Solución AtomicInteger, synchronized o un contador en DB / servletContext.

Sesión 7 ㊶

SQLException de tipo communicationsException (o unable to connect to host / connection refused).
Por qué: el driver intenta conectarse al puerto 3307, donde no hay nada escuchando. MySQL está en 3306, la conexión se rechaza antes de autenticar, por eso no es Access denied sino de error conexión.

Sesión 7 ㊷

SQLException: Data conversion error / cannot convert value ("mendoza") carlos mendoza to integer, porque rs.getInt(1) intenta convertir el texto "carlos mendoza" a int, el driver no puede parsearlo y lanza excepción. Para leer un texto hay que usar rs.getString(1)